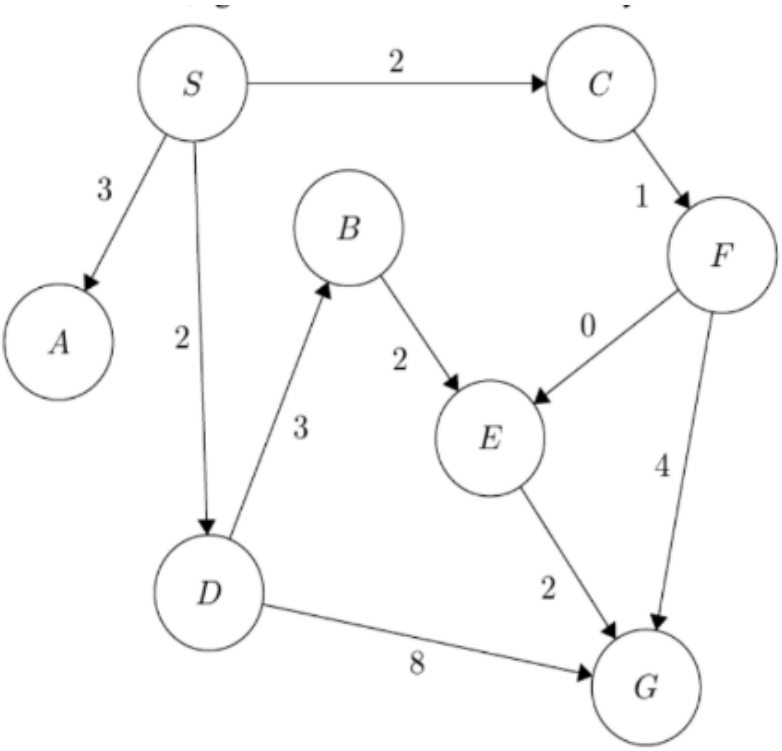
BÀI TẬP TÌM KIẾM TRONG AI

Họ và tên	Phan Thanh Tùng
MSSV/Lớp	3123410417

Bài 1. Các thuật toán tìm kiếm



1.1. Breadth-First Search (BFS)

Nội dung	Kết quả
Explored Set	S,A,C,D,F,B,G
Path	S -> D -> G
Total Cost	2 + 8 = 10

1.2. Depth-First Search (DFS)

Nội dung	Kết quả
Explored Set	S,A,C,D,F,E,G

Path	S -> C -> F -> E -> G
Total Cost	2 + 1 + 0 + 2 = 5

1.3. Iterative Deepening Search (IDS)

Giới hạn độ sâu	Các nút được duyệt
0	S
1	S, A, C, D
2	S, A, C, F, D, B, G

Giải thích vòng lặp độ sâu 2:

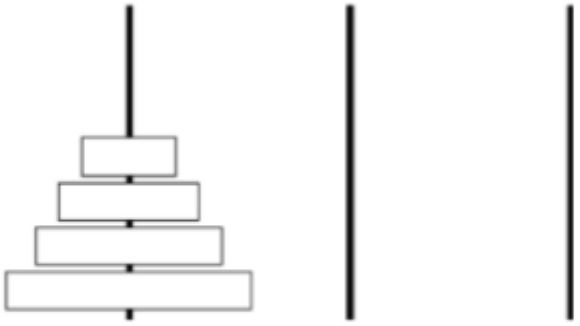
- Gốc S (độ sâu 0)
- Nhánh A: duyệt $S \rightarrow A$ (hết độ sâu 1, không có nút con).
- Nhánh C: duyệt $S \rightarrow C \rightarrow F$ (độ sâu 2).
- Nhánh D: duyệt $S \rightarrow D \rightarrow B$ (độ sâu 2), tiếp tục nhánh kế tiếp của D là G (độ sâu 2 → tìm thấy đích G).

Path cuối cùng: S → D → G
Total Cost: 2 + 8 = 10

Câu hỏi thảo luận: BFS và DFS khác nhau ở cấu trúc frontier và tính tối ưu như thế nào?

- **Cấu trúc Frontier (Tập biên):**
  - BFS: Sử dụng hàng đợi Queue (FIFO - First In, First Out), nút nào vào trước sẽ được lấy ra mở rộng trước, giúp mở rộng đều theo từng mức độ sâu.
  - DFS: Sử dụng ngăn xếp Stack (LIFO - Last In, First Out), nút nào vào sau cùng sẽ được lấy ra mở rộng trước, giúp đi sâu xuống một nhánh đến khi gặp ngõ cụt rồi mới quay lui.
- **Tính tối ưu (Optimality):**
  - BFS: Tối ưu về số bước đi / số cạnh (luôn tìm ra đường đi có ít cạnh nhất từ trạng thái xuất phát đến trạng thái đích). Tuy nhiên, BFS không tối ưu về tổng chi phí khi đồ thị có trọng số đường đi khác nhau (như bài này BFS ra chi phí 10 trong khi DFS ra chi phí 5).
  - DFS: Không đảm bảo tính tối ưu cả về số bước đi lẫn tổng chi phí đường đi, vì nó chỉ tìm và trả về đường đi đầu tiên nó chạm tới đích mà không so sánh với các nhánh khác.

Bài 2. Tháp Hà Nội - Tower of Hanoi



Hình 2. Trạng thái ban đầu của bài toán Tháp Hà Nội

2.1. Biểu diễn trạng thái

- Gọi 3 cọc là **cọc 1, cọc 2, cọc 3** (hoặc A, B, C).
- Đánh số các đĩa từ **1 đến n theo thứ tự từ nhỏ nhất đến lớn nhất** (đĩa 1 nhỏ nhất, đĩa n lớn nhất).
- **Cách biểu diễn:** Dùng một danh sách gồm n phần tử, trong đó vị trí thứ i cho biết **đĩa i hiện đang nằm ở cọc nào**.
    - Ví dụ với 3 đĩa: Trạng thái (1, 1, 1) nghĩa là cả 3 đĩa đều đang ở cọc 1; trạng thái (2, 3, 1) nghĩa là đĩa 1 ở cọc 2, đĩa 2 ở cọc 3, đĩa 3 ở cọc 1.
- **Vì sao cách này đủ xác định vị trí?**
    - Do luật chơi quy định không được đặt đĩa lớn lên trên đĩa nhỏ, nên trên bất kỳ cọc nào, các đĩa luôn tự động được xếp theo thứ tự đĩa nhỏ nằm trên, đĩa lớn nằm dưới. Vì vậy, ta chỉ cần biết đĩa đang ở cọc nào là hoàn toàn biết được chính xác thứ tự xếp của nó mà không sợ bị nhầm lẫn.

2.2. Kích thước không gian trạng thái

Số trạng thái có thể có: 3^n trạng thái.

Giải thích:

- Có tất cả n đĩa.
- Mỗi một đĩa đều có thể nằm độc lập ở 1 trong 3 cọc (cọc 1, cọc 2 hoặc cọc 3), tức là mỗi đĩa có 3 sự lựa chọn.

- Vì trên mỗi cọc thứ tự các đĩa luôn cố định (đĩa nhỏ hơn bắt buộc phải ở trên đĩa lớn hơn), nên cứ mỗi cách phân chia n đĩa vào 3 cọc sẽ tạo ra đúng 1 trạng thái hợp lệ duy nhất.
- Lấy 3 lựa chọn nhân cho từng đĩa: 3 * 3 * … * 3 (n lần) = 3^n trạng thái.

2.3. Trạng thái bắt đầu

- Ban đầu, tất cả n đĩa đều đang được xếp gọn gàng ở cọc đầu tiên (cọc 1).
- **Trạng thái khởi đầu:** Tất cả các vị trí đều bằng 1:
    - Trạng thái bắt đầu là: (1, 1, ..., 1) (gồm n số 1).

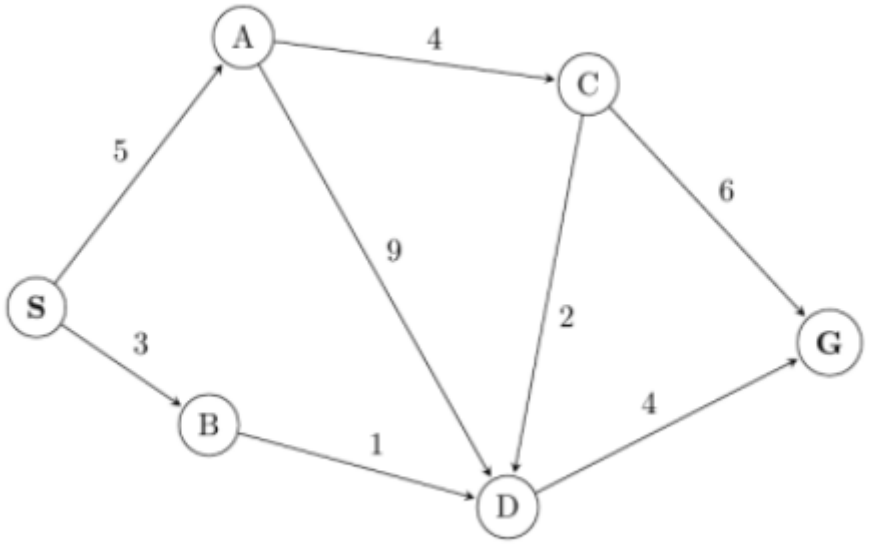
2.4. Các hành động hợp lệ

- **Mô tả hành động:** Di chuyển 1 đĩa từ cọc này sang cọc khác (kí hiệu: *Chuyển đĩa từ cọc nguồn sang cọc đích*).
- **Một hành động được coi là hợp lệ khi thỏa mãn đồng thời 3 điều kiện:**
    1. **Cọc nguồn phải có đĩa** (không được bốc đĩa từ cọc đang trống).
    2. **Chỉ được bốc đĩa nằm ở trên cùng** của cọc nguồn (đây chính là đĩa có kích thước nhỏ nhất hiện có trên cọc đó).
    3. **Đặt đĩa vào cọc đích hợp lệ:** Cọc đích phải đang trống, **hoặc** nếu cọc đích đã có đĩa thì đĩa chuyển sang phải có kích thước nhỏ hơn đĩa đang nằm trên cùng của cọc đích (đảm bảo không bao giờ đặt đĩa to lên trên đĩa nhỏ).

2.5. Goal Test (Kiểm tra trạng thái đích)

- **Mục tiêu:** Chuyển thành công toàn bộ n đĩa từ cọc 1 sang cọc 3.
- **Mô tả hàm goalTest(state):**
    - Hàm sẽ trả về **True** khi **tất cả các đĩa đều đang nằm ở cọc 3**, tức là trạng thái hiện tại là: (3, 3, ..., 3) (gồm n số 3).
    - Nếu còn bất kỳ đĩa nào chưa nằm ở cọc 3, hàm sẽ trả về **False**.

Bài 3. Phân tích và thiết kế Heuristic



Hình 3. Đồ thị dùng để thiết kế heuristic

Công thức A*: f(n) = g(n) + h(n), trong đó g(n) là chi phí thực tế từ S đến n và h(n) là chi phí ước lượng từ n đến đích.

3.1. Thiết kế heuristic admissible và consistent

Một heuristic $h(n)$ được gọi là:

1. **Admissible (chấp nhận được):** nếu $0 \leq h(n) \leq h^*(n)$ với mọi nút n, trong đó $h^*(n)$ là chi phí thực tế ngắn nhất từ n đến đích G.
2. **Consistent (nhất quán / đơn điệu):** nếu với mọi cung $n \to n'$ có chi phí $c(n, n')$, ta luôn có $h(n) \leq c(n, n') + h(n')$ và $h(G) = 0$.

Cách tự nhiên và chuẩn nhất để đảm bảo tính **consistent** (và do đó admissible) là chọn $h(n)$ chính bằng chi phí đường đi ngắn nhất thực tế từ n đến G ($h(n) = h^*(n)$):

- **G:** $h(G) = 0$
- **D:** Đường đi $D \to G$ có chi phí là $\Rightarrow h(D) = 4$
- **C:**
  - $C \to G$ chi phí 6
  - $C \to D \to G$ chi phí $2 + 4 = 6$
    $\Rightarrow h(C) = 6$
- **B:** Đường đi $B \to D \to G$ có chi phí 1 + 4 = 5 $\Rightarrow h(B) = 5$
- **A:**
  - $A \to C \to G$ chi phí $4 + 6 = 10$

- $A \rightarrow D \rightarrow G$ chi phí $9 + 4 = 13$
  $=> h(A) = 10$

- **S:**
  - $S -> A -> \ldots => 5 + 10 = 15$
  - $S -> B -> D -> G = 3 + 1 + 4 = 8$

    $=> h(S) = 8$

Bảng giá trị

Node	h(n)
S	8
A	10
B	5
C	6
D	4
G	0

3.2. Chạy A* với heuristic đã thiết kế

Áp dụng công thức $f(n) = g(n) + h(n)$:

1. **Khởi tạo:**
   - Tập biên (Frontier): $\{S(g = 0, f = 0 + 8 = 8)\}$

2. **Bước 1:**
   - Lấy S ($f = 8$) ra mở rộng => Thêm S vào Explored Set.
   - Các nút kề từ S:
     - $A: g(A) = 5, f(A) = 5 + 10 = 15$
     - $B: g(B) = 3, f(B) = 3 + 5 = 8$
   - Frontier: $\{B(f = 8), A(f = 15)\}$
3. **Bước 2:**
   - Chọn nút có f nhỏ nhất là B ($f = 8$) ra mở rộng => Thêm B vào Explored Set.
   - Nút kề từ B:
     - D: g(D) = g(B) + c(B, D) = 3 + 1 = 4 => f(D) = 4 + 4 = 8
   - Frontier: $\{D(f = 8), A(f = 15)\}$
4. **Bước 3:**

- Chọn nút có f nhỏ nhất là D ($f = 8$) ra mở rộng => Thêm D vào Explored Set.
- Nút kề từ D:
  - G: g(G) = g(D) + c(D, G) = 4 + 4 = 8 => f(G) = 8 + 0 = 8

- Frontier: $\{G(f = 8), A(f = 15)\}$

5. **Bước 4:**
   - Chọn nút có f nhỏ nhất là G ($f = 8$) ra mở rộng => Thêm G vào Explored Set (đã chạm đích). Dừng thuật toán.

Kết quả điền vào form:

- **Explored Set:** S, B, D, G
- **Path:** S → B → D → G
- **Total Cost:** 8

3.3. Heuristic admissible nhưng không consistent

- **Đề xuất bộ giá trị $h(n)$:**
  - Giữ nguyên các nút khác: $h(S) = 8, h(B) = 5, h(D) = 4, h(G) = 0$.
  - Thay đổi giá trị của C: cho $h(C) = 1$ (thay vì 6).
- **Kiểm tra tính Admissible:**
  - Vì chi phí thực ngắn nhất từ C đến G là 6, việc chọn $h(C) = 1 \leq 6$ vẫn đảm bảo không đánh giá vượt quá chi phí thực tế => Admissible.
- **Chỉ ra cạnh vi phạm điều kiện Consistent ($h(n) \leq c(n, n') + h(n')$):**
  - Xét cạnh $A \rightarrow C$ có chi phí $c(A, C) = 4$:
    - $h(A) = 10$
    - $c(A, C) + h(C) = 4 + 1 = 5$
  - Ta thấy: $10 > 5$, tức là $h(A) > c(A, C) + h(C)$ => Vi phạm tính chất consistent.

3.4. Câu hỏi mở rộng

Đề bài: Giải thích vì sao một heuristic consistent thì cũng admissible, giả sử $h(G) = 0$.

Giải thích:

1. **Hiểu bản chất của điều kiện Consistent (bất đẳng thức tam giác):**
   - Tính chất consistent quy định rằng: Khi bạn bước một bước từ nút hiện tại sang nút kế tiếp, giá trị ước lượng h không thể giảm nhanh hơn chi phí thực tế mà bạn vừa bỏ ra để đi bước đó.

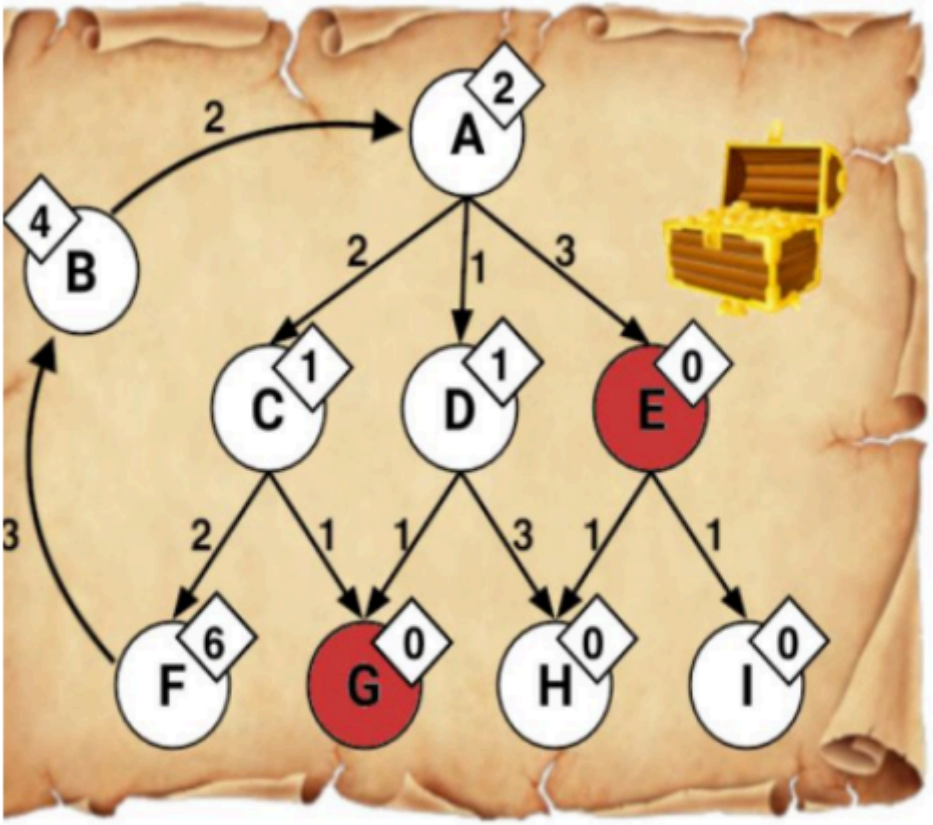
- Nói cách khác: Độ sụt giảm của h qua mỗi bước luôn nhỏ hơn hoặc bằng chi phí thực tế của bước đi đó.

2. **Mở rộng cho cả đoạn đường:**
   - Giả sử ta đi từ nút n bất kỳ về đến đích G theo con đường ngắn nhất thực tế.
   - Tổng độ sụt giảm của h từ đầu đường (tại nút n) cho đến cuối đường (tại nút đích G) sẽ bằng:

     $$h(n) - h(G)$$

   - Vì đề bài cho $h(G) = 0$, nên tổng độ sụt giảm giá trị h trên cả quãng đường chính là $h(n)$.
3. **So sánh với chi phí thực tế:**
   - Do ở mỗi bước đi riêng lẻ, độ sụt giảm của h không bao giờ vượt quá chi phí của bước đó, nên khi cộng dồn lại trên toàn bộ hành trình, tổng độ sụt giảm của h chắc chắn không thể lớn hơn tổng chi phí thực tế ngắn nhất để đi từ n về G.
   - Tức là: giá trị ước lượng $h(n)$ luôn nhỏ hơn hoặc bằng chi phí thực tế ngắn nhất ($h(n) \leq h^*(n)$).
4. **Kết luận:**
   - Vì $h(n)$ không bao giờ ước lượng cao hơn chi phí thực tế, nên nó thỏa mãn đúng định nghĩa của admissible (chấp nhận được). Do đó, mọi heuristic consistent (với $h(G) = 0$) đều đương nhiên là admissible

Bài 4. Treasure Hunting (Tìm kho báu)



Hình 4. Bản đồ tìm kho báu (giữ nguyên hình gốc)

Quy tắc phá hòa: khi lấy phần tử khỏi frontier, nếu hòa thì chọn theo thứ tự alphabet.

4.1. Depth-First Search (DFS)

- **Quá trình thực hiện (Ngăn xếp - LIFO):**
  - Từ A, sinh ra các nhánh con: C, D, E. Để lấy ra theo thứ tự alphabet ($C \rightarrow D \rightarrow E$), các nút được đẩy vào Stack sao cho C nằm trên cùng (đẩy E, D, rồi C).
  - Lấy A ra mở rộng → Explored: $\{A\}$.
  - Lấy C ra mở rộng → Explored: $\{A, C\}$. Sinh ra nhánh con từ C: F, G.
  - Để ưu tiên F trước G theo alphabet: lấy F ra mở rộng → Explored: $\{A, C, F\}$.
  - Từ F đi đến B. Lấy B ra mở rộng → Explored: $\{A, C, F, B\}$.
  - Từ B đi đến A (đã duyệt, bỏ qua).
  - Quay lui (lấy tiếp từ stack): Lấy G ra khỏi Frontier → G là nút đích (kho báu), dừng thuật toán.

- **Explored Set:** A, C, F, B, G *(hoặc A, C, F, B nếu dừng ngay khi lấy G ra kiểm tra)*
- **Path returned:** A → C → G
- **Cost:** $2 + 1 = 3$

4.2. Breadth-First Search (BFS)

- **Quá trình thực hiện (Hàng đợi - FIFO):**
  - Mức 0: Bắt đầu tại A.
  - Khám phá A: đưa các nút kề vào hàng đợi theo alphabet: C, D, E.
  - Khám phá C: đưa F, G vào hàng đợi. Hàng đợi hiện tại: $[D, E, F, G]$.
  - Khám phá D: đưa H vào hàng đợi (G đã có trong frontier). Hàng đợi: $[E, F, G, H]$.
  - Lấy E ra kiểm tra → E là trạng thái đích (kho báu) => Dừng thuật toán.
- **Explored Set:** A, C, D, E *(hoặc A, C, D)*
- **Path returned:** A → E
- **Cost:** 3

4.3. Uniform-Cost Search (UCS)

UCS duyệt dựa trên chi phí đường đi thực tế tích lũy $g(n)$ nhỏ nhất:

1. Khởi tạo: Frontier = $\{A(g = 0)\}$.
2. Lấy A ($g = 0$) → Explored: $\{A\}$.
   - Sinh: $D(g = 1), C(g = 2), E(g = 3)$.
3. Lấy D ($g = 1$) → Explored: $\{A, D\}$.
   - Sinh từ D: $G(g = 1 + 1 = 2), H(g = 1 + 3 = 4)$.
   - Frontier: $\{C(g = 2), G(g = 2), E(g = 3), H(g = 4)\}$.
4. So sánh C và G cùng có $g = 2$: Phá hòa alphabet → Lấy C.
   - Explored: $\{A, D, C\}$.
   - Sinh từ C: G (đã có đường đi với $g = 2$, không cải thiện), $F(g = 2 + 2 = 4)$.
   - Frontier: $\{G(g = 2), E(g = 3), F(g = 4), H(g = 4)\}$.
5. Lấy G ($g = 2$) → G là đích => Dừng thuật toán.

- **Explored Set:** A, D, C, G *(hoặc A, D, C)*
- **Path returned:** A → D → G
- **Cost:** 2

4.4. Greedy Search

Greedy Search duyệt dựa trên giá trị heuristic $h(n)$ nhỏ nhất:

1. Khởi tạo: Frontier = $\{A(h = 2)\}$.

2. Lấy A ($h = 2$) → Explored: $\{A\}$.
    - Sinh: $C(h = 1), D(h = 1), E(h = 0)$.
    - Frontier: $\{E(h = 0), C(h = 1), D(h = 1)\}$.
3. Lấy E có $h = 0$ nhỏ nhất → E là đích => Dừng thuật toán.

- **Explored Set:** A, E *(hoặc A)*
- **Path returned:** A → E
- **Cost:** 3

4.5. A* Search

A* duyệt dựa trên hàm đánh giá $f(n) = g(n) + h(n)$:

1. Khởi tạo: Frontier = {A (g=0, h=2 => f=2)}.
2. Lấy A ($f = 2$) → Explored: $\{A\}$.
    - Các nút kề từ A:
        - D: g=1, h=1 => f = 2
        - C: g=2, h=1 => f = 3
        - E: g=3, h=0 => f = 3
    - Frontier: $\{D(f = 2), C(f = 3), E(f = 3)\}$.
3. Lấy D ($f = 2$) → Explored: $\{A, D\}$.
    - Các nút kề từ D:
        - G: g = 1 + 1 = 2, h=0 => f = 2
        - H: g = 1 + 3 = 4, h=0 => f = 4
    - Frontier: $\{G(f = 2), C(f = 3), E(f = 3), H(f = 4)\}$.
4. Lấy G ($f = 2$) → G là đích => Dừng thuật toán.

- **Explored Set:** A, D, G *(hoặc A, D)*
- **Path returned:** A → D → G
- **Cost: 2**

Bảng tổng hợp so sánh

Thuật toán PNG	Tiêu chí chọn node PNG	Path PNG	Cost PNG
DFS	Độ sâu	A -> C -> G	3
BFS	Độ sâu nhỏ nhất	A -> E	3
UCS	g(n)	A -> D -> G	2
Greedy	h(n)	A -> E	3
A*	g(n) + h(n)	A -> D -> G	2

Bài 5. Câu hỏi Đúng/Sai

Câu 1. Depth-first search luôn mở rộng ít nhất nhiều node bằng A* sử dụng một admissible heuristic.

- **Đúng / Sai**: Sai
- **Giải thích:** DFS có thể "may mắn" chọn đúng ngay nhánh dẫn thẳng đến đích mà không phải khám phá các nhánh khác, khi đó số node DFS mở rộng có thể ít hơn A*. A* mở rộng tất cả các node có $f(n) < C^*$ để đảm bảo tính tối ưu, nên số node A* duyệt có thể nhiều hơn DFS trong trường hợp này.

Câu 2. Giả sử một quân xe trong cờ vua có thể đi bất kỳ số ô theo hàng hoặc cột trong một nước đi và không thể nhảy qua quân khác. Manhattan distance là một admissible heuristic cho số nước đi nhỏ nhất từ ô A đến ô B.

- **Đúng / Sai:** Sai
- **Giải thích:** Heuristic admissible không được ước lượng lớn hơn chi phí thực tế ($h \leq h^*$). Khoảng cách Manhattan tính số ô di chuyển từng bước đơn lẻ, trong khi quân xe có thể đi qua nhiều ô chỉ trong 1 nước đi.
    - *Phản ví dụ:* Quân xe đi từ ô $(1, 1)$ đến ô $(1, 8)$ trên bàn cờ trống chỉ mất 1 nước đi ($h^* = 1$). Tuy nhiên, khoảng cách Manhattan là | 1 - 1 | + | 8 - 1 | = 8. Vì 7 > 1, heuristic đánh giá vượt quá chi phí thực tế nên không admissible.

Câu 3. Euclidean distance là một admissible heuristic cho bài toán tìm đường của Pacman.

- **Đúng / Sai:** Đúng
- **Giải thích:** Khoảng cách Euclidean là độ dài đoạn thẳng nối trực tiếp giữa hai điểm trong mặt phẳng 2D. Trong bài toán Pacman, các bước đi chỉ theo trục dọc/ngang (khoảng cách Manhattan) và thường có thêm vật cản tường. Vì đường thẳng nối trực tiếp luôn là đường ngắn nhất giữa 2 điểm bất kỳ ($h_{Euclidean} \leq h^*$), nên nó không bao giờ vượt quá chi phí thực tế và do đó là admissible.

Câu 4. Tổng của nhiều admissible heuristics vẫn luôn là một admissible heuristic.

- **Đúng / Sai:** Sai
- **Giải thích:** Khi cộng hai hay nhiều heuristic admissible lại với nhau, tổng của chúng rất dễ vượt quá chi phí tối ưu thực tế h^*. *(Chỉ có phép lấy giá trị lớn nhất* $\max(h_1, h_2)$ *mới chắc chắn giữ được tính admissible).*

- *Phản ví dụ:* Giả sử chi phí thực tế $h^* = 5$. Ta chọn hai heuristic admissible là $h_1 = 3$ và $h_2 = 4$. Tổng $h = h_1 + h_2 = 7 > h^* = 5$, vi phạm điều kiện admissible.

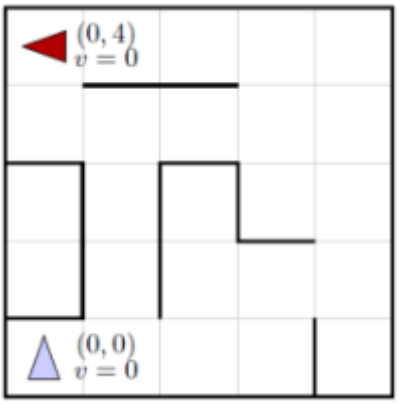
Câu 5. Admissibility của một heuristic cho A* kéo theo consistency.

- **Đúng / Sai:** Sai
- **Giải thích:** Tính chất consistent kéo theo admissible (như đã chứng minh ở Bài 3.4), nhưng chiều ngược lại không đúng. Một heuristic có thể không bao giờ vượt quá chi phí thực tế (admissible) nhưng vẫn có thể vi phạm bất đẳng thức tam giác giữa các node kề nhau.
  - *Phản ví dụ:* Xét đồ thị $S \to A \to G$ với $c(S, A) = 1, c(A, G) = 4$. Chi phí thực: $h^*(S) = 5, h^*(A) = 4, h^*(G) = 0$. Nếu đặt $h(S) = 4, h(A) = 1, h(G) = 0$, ta có $h \leq h^*$ trên mọi node (admissible). Nhưng xét cung (S, A): $h(S) = 4 > c(S, A) + h(A) = 1 + 1 = 2$, nên không consistent.

Câu 6. A* với graph search luôn tối ưu nếu heuristic admissible.

- **Đúng / Sai:** Sai
- **Giải thích:** Với Tree Search, A* chỉ cần điều kiện heuristic admissible là đảm bảo tìm ra đường đi tối ưu. Tuy nhiên, với Graph Search (có sử dụng tập explored để không duyệt lại các node đã thăm), A* đòi hỏi heuristic phải thỏa mãn tính chất consistent (đơn điệu) thì mới luôn đảm bảo tìm ra nghiệm tối ưu. Nếu heuristic chỉ admissible mà không consistent, A* graph search có thể đóng một node trước khi tìm thấy đường đi rẻ hơn đến node đó, dẫn tới kết quả không tối ưu.

Bài 6. Tìm kiếm và heuristic



a) Giả sử tác tử đi theo nhánh ngoài cùng bên trái (đi qua ô (1,2)), từ trạng thái bắt đầu (0,0), hướng Bắc đến trạng thái đích (0,4), hướng Tây. Hãy viết chuỗi hành động ngắn nhất.

=> Tăng tốc, tăng tốc, giảm tốc, giảm tốc, rẽ trái

b) Nếu lưới có kích thước M × N và vận tốc tối đa là Vmax, kích thước không gian trạng thái là bao nhiêu? Giả sử mọi cấu hình đều có thể đạt được từ trạng thái bắt đầu. Một trạng thái được xác định bởi bộ 4 thông số: (x, y, hướng, vận tốc).

- Vị trí (x, y): có $M \times N$ ô.
- Hướng $d \in \{Bắc, Nam, Đông, Tây\}$: có 4 hướng.
- Vận tốc v ∈ {0, 1, 2, … , V_{max}}$: có $V_{max} + 1$ giá trị vận tốc.

Kích thước không gian trạng thái:

$$M \times N \times 4 \times (V_{max} + 1)$$

c) Một "nút con" của trạng thái s là trạng thái s' có thể đạt được bằng một hành động hợp lệ từ s. Có thể tồn tại trạng thái không có nút con hay không? Nếu có, hãy cho ví dụ; nếu không, hãy giải thích.

☐ Có ☐ Không

Có

Giải thích / Ví dụ:

- Ví dụ: Xét trạng thái xe đang có vận tốc $v \geq 1$ và ngay trước mặt là một bức tường cản (khoảng cách tới tường nhỏ hơn v). Khi đó:
  - Không thể rẽ vì điều kiện rẽ bắt buộc $v = 0$.
  - Nếu chọn di chuyển (dù tăng tốc, giữ tốc hay giảm tốc với $v' = v - 1 \geq 0$), quãng đường đi tiếp vẫn sẽ đâm thẳng vào tường → vi phạm quy tắc va vào tường.
  - Do đó, mọi hành động xuất phát từ trạng thái này đều không hợp lệ, tức là trạng thái không có nút con nào.

d) Hệ số phân nhánh lớn nhất của bài toán là bao nhiêu? Hãy cho một trạng thái ví dụ (x, y, hướng, vận tốc) đạt hệ số phân nhánh này và liệt kê các hành động có thể thực hiện.

Hệ số phân nhánh lớn nhất: 3

Trạng thái ví dụ:

- Xét trạng thái xe đang dừng: $(x, y, hướng, v = 0)$ ở một ô thoáng, phía trước không có tường cản (ví dụ ô $(0, 0)$, hướng Bắc, $v = 0$).

Liệt kê các hành động có thể thực hiện (đúng 3 hành động):

1. Rẽ trái (hợp lệ vì $v = 0$)
2. Rẽ phải (hợp lệ vì $v = 0$)
3. Tăng tốc (chuyển $v' = 1$ và tiến lên phía trước 1 ô) (Lưu ý: Không thể giảm tốc vì v không được < 0; không thể giữ tốc vì đề bài cấm giữ tốc độ khi $v = 0$. Khi $v > 0$, xe chỉ có tối đa 3 lựa chọn di chuyển: Tăng tốc, Giữ tốc, Giảm tốc và không được rẽ).

e) Khoảng cách Manhattan từ vị trí hiện tại của tác tử đến vị trí cửa ra có phải là heuristic chấp nhận được không? Nếu không, hãy cho một trạng thái phản ví dụ, nêu giá trị heuristic và chi phí thực tế tối ưu đến đích.

☐ Có ☐ Không

Không

Giải thích và phản ví dụ:

- Heuristic admissible bắt buộc phải luôn nhỏ hơn hoặc bằng chi phí thực tế tối ưu ($h \leq h^*$). Ở đây, mỗi hành động là 1 bước chi phí, nhưng trong 1 bước di chuyển với vận tốc lớn ($v > 1$), xe có thể vượt qua nhiều ô cùng lúc, làm cho số hành động thực tế có thể nhỏ hơn khoảng cách Manhattan.
- Phản ví dụ: Giả sử xe đang ở vị trí $(0, 0)$, hướng Bắc, với vận tốc hiện tại $v = 3$ đang chạy thẳng tới đích ở $(0, 4)$.
    - Khoảng cách Manhattan: | 0 - 0 | + | 4 - 0 | = 4 => h = 4
    - Nhưng nếu từ đây xe chỉ cần thực hiện 1 hành động giữ tốc $v' = 3$ hoặc hành động phù hợp để đi qua quãng đường dài chỉ trong ít bước hơn, thì chi phí thực tế có thể nhỏ hơn 4.
    - Rõ ràng nhất: Ở câu (a), đi quãng đường Manhattan = 4 chỉ cần 4 hành động di chuyển (nếu đích cho phép hướng thẳng), hoặc xét trường hợp V_{max} lớn, xe có thể nhảy vọt nhiều ô chỉ với 1 hành động. Do đó khoảng cách Manhattan có thể lớn hơn số bước thực tế => Không admissible.

f) Heuristic h = khoảng cách Manhattan / Vmax có phải là admissible không? Giải thích. Nếu không, hãy cho phản ví dụ tương tự câu (e).

☐ Có ☐ Không

Có

Giải thích:

- Trong mỗi hành động di chuyển, xe đi được quãng đường tối đa là V_{max} ô (khi chạy ở tốc độ cao nhất).
- Để vượt qua tổng khoảng cách vị trí ít nhất là khoảng cách Manhattan (khoảng cách tối thiểu trên lưới không có vật cản), số hành động di chuyển chắc chắn phải lớn hơn hoặc bằng:
    - $\frac{KhoảngcáchManhattan}{V_{max}}$
- Hơn nữa, chi phí thực tế còn bắt buộc phải cộng thêm các thao tác tăng tốc, giảm tốc về 0 và các lượt rẽ để đúng hướng đích. Vì vậy, giá trị $\frac{KhoảngcáchManhattan}{V_{max}}$ không bao giờ vượt quá chi phí thực tế tối ưu => Là admissible

g) Nếu dùng heuristic không admissible trong A* Tree Search, tính đầy đủ của thuật toán có thể thay đổi không? Giả sử đồ thị hữu hạn và heuristic không âm.

☐ Có ☐ Không

Không

Giải thích:

- Giả thiết đề bài cho đồ thị hữu hạn và heuristic không âm ($h(n) \geq 0$ hữu hạn). Chi phí mỗi bước đi đều dương ($c \geq 1$).
- Vì không gian trạng thái hữu hạn và không có chu trình chi phí âm/bằng 0 vô tận, nếu tồn tại nghiệm (đích đến được), thuật toán A* vẫn sẽ duyệt qua và cuối cùng tìm thấy đích chứ không bị kẹt hay treo vĩnh viễn => Tính đầy đủ (completeness) không thay đổi (vẫn đầy đủ).

h) Nếu dùng heuristic không admissible trong A* Tree Search, tính tối ưu của thuật toán có thể thay đổi không? Giả sử đồ thị hữu hạn và heuristic không âm.

☐ Có ☐ Không

Có

Giải thích:

- Khi heuristic không admissible, nó có thể đánh giá quá cao (overestimate) chi phí của một nút dẫn tới đường đi tối ưu thật sự.

- Điều này khiến giá trị $f(n) = g(n) + h(n)$ của đường đi tối ưu bị đội lên rất cao, làm cho thuật toán A* ưu tiên chọn mở rộng và kết thúc tại một trạng thái đích theo con đường khác kém tối ưu hơn trước $\implies$ Tính tối ưu bị mất đi.

i) Những lý do nào sau đây có thể hợp lý khi dùng heuristic không admissible thay vì admissible?

Tích chọn các phương án:

- [x] Heuristic không admissible có thể dễ tính hơn, giúp giảm thời gian tính heuristic. (Đúng, giảm tải tài nguyên tính toán).
- [x] Heuristic không admissible có thể gần chi phí thực hơn (dù đôi khi đánh giá cao), nhờ đó có thể mở ít nút hơn. (Đúng, giúp thuật toán hướng thẳng về đích nhanh hơn rất nhiều trên thực tế).
- [ ] Heuristic không admissible vẫn luôn tìm được đường đi tối ưu khi mọi chi phí thực không âm. (Sai, đã khẳng định ở câu h).
- [ ] Heuristic không admissible có thể được dùng để làm thuật toán hoàn toàn không xét một phần của đồ thị. (Không phù hợp làm định nghĩa lý do chuẩn của việc chọn heuristic không admissible).

Bài 7. Các nút tìm kiếm

- Ký hiệu chuẩn:
  - n_c: nút đích có chi phí đường đi nhỏ nhất (tức chi phí tối ưu là $C^* = g(n_c)$).
  - n_s: nút đích ở độ sâu nông nhất (tức độ sâu tối thiểu là $d(n_s)$).

a) Breadth-First Search (BFS)

- BFS duyệt tuần tự theo từng lớp độ sâu, chỉ dừng lại khi chạm tới đích nông nhất n_s. Tất cả các nút được mở rộng trước đó bắt buộc phải nằm ở các tầng có độ sâu nhỏ hơn độ sâu của nút đích nông nhất:
  - **Bất đẳng thức:**
  - $d(n) < d(n_s)$

b) Uniform-Cost Search (UCS)

- UCS luôn ưu tiên lấy nút có chi phí đường đi tích lũy $g(n)$ nhỏ nhất để mở rộng, và dừng lại khi lấy nút đích tối ưu n_c ra khỏi hàng đợi ưu tiên. Do đó, tất cả các nút được mở rộng trước khi kết thúc đều phải có chi phí:
    - **Bất đẳng thức:**
    - $g(n) < g(n_c)$
    - *(hoặc viết là $g(n) < C^*$)*

c) A* Search, giả sử h là heuristic nhất quán (consistent)

- A* mở rộng các nút theo giá trị $f(n) = g(n) + h(n)$ tăng dần. Thuật toán chỉ kết thúc khi lấy nút đích tối ưu n_c ra khỏi hàng đợi (với $f(n_c) = g(n_c) + h(n_c) = g(n_c) = C^*$ vì $h(n_c) = 0$). Do đó, mọi nút được mở rộng chắc chắn phải thỏa mãn:
    - **Bất đẳng thức:**
    - $g(n) + h(n) < g(n_c)$
    - *(hoặc viết là f(n) < C* hay $g(n) + h(n) < C^*$)*

Bài 8. Tìm kiếm trên đồ thị

a) Bảng đánh dấu đường đi của các chiến lược tìm kiếm

- **UCS:** Luôn tìm ra đường đi có chi phí nhỏ nhất là 11 (đường A-C-E-F-G).
- **Greedy với h_1:**
    - Từ A (các con: B có $h_1 = 6$, C có $h_1 = 9$) → chọn B vì h_1 nhỏ nhất.
    - Từ B (các con: C có $h_1 = 9$, D có $h_1 = 3$, E có $h_1 = 3$). Hòa giữa D và E ($h_1 = 3$):
        - Nếu chọn E: từ E con là $F(h_1 = 2), G(h_1 = 0)$ → đến G. Tạo thành đường: A-B-E-G (thuộc cột Khác).
        - Nếu chọn D: từ D đến E, rồi từ E đến G hoặc qua F → có thể ra A-B-D-E-F-G hoặc A-B-D-E-G.
        - Do đó có thể trả về A-B-D-E-F-G hoặc Khác.
- **Greedy với h_2:**
    - Từ A: C có $h_2 = 6$, B có $h_2 = 7$ → chọn C.
    - Từ C: con $B(h_2 = 7), E(h_2 = 5)$ → chọn E.
    - Từ E: con $D(h_2 = 4), F(h_2 = 1), G(h_2 = 0)$ → chọn ngay G ($h_2 = 0$).

- Thu được đường: A-C-E-G.

A* với h_1:

- Từ A:
  - Đến B: g=3, h_1=6 => f = 9
  - Đến C: g=4, h_1 = 9 => f = 13

    → Mở rộng B.
- Từ B:
  - Đến D: g = 6, h_1=3 => f = 9
  - Đến E: g = 8, h_1=3 => f = 11
  - Đến C: g = 5, h_1=9 => f = 14
- Lấy D ($f = 9$): đến E với g=7, h_1= 3 => f = 10
- Lấy E ($f = 10$):
  - Đến F: g=10, h_1=2 => f=12
  - Đến G: g=13, h_1=0 => f=13
- Hàng đợi có: E qua nhánh khác ($f = 11$), F ($f = 12$), G ($f = 13$), C ($f = 13$).
- Lấy F ($f = 12$): đến G với g=12, h_1=0 => f=12.
- Lấy G ($f = 12$): Dừng, trả về đường A-B-D-E-F-G.

A* với h_2:

- Vì h_2 là admissible và A* duyệt thì nhánh qua C tối ưu có $f(A) = 0 + 11 = 11$.
- Đến C: g=4, h_2=6 => f=10.
- Đến E: g=6, h_2=5 => f=11.
- Đến F: g=9, h_2=1 => f=10.
- Đến G: g=11, h_2=0 => f=11.
- Đường đi tìm được là đường tối ưu: A-C-E-F-G.

Thuật toán	**A-C-E-G**	**A-C-E-F-G**	**A-B-D-E-F-G**	**Khác**

UCS		**[X]**		
Greedy với h1			**[X]**	**[X]** *(A-B-E-G)*
Greedy với h2	**[X]**			
A* với h1			**[X]**	
A* với h2		**[X]**		

b) Chi phí của đường đi tối ưu từ A đến G theo UCS là bao nhiêu?

- **Đáp án: 11** (đường đi tối ưu là A-C-E-F-G có chi phí $4 + 2 + 3 + 2 = 11$).

c) Heuristic h_1 có admissible không? Có consistent không?

- So sánh $h_1(n)$ với chi phí tối ưu thực tế $h^*(n)$:
  - $A: h_1(A) = 12 > h^*(A) = 11$
  - $C: h_1(C) = 9 > h^*(C) = 7$
- Vì $h_1(A) > h^*(A)$ và $h_1(C) > h^*(C)$ nên heuristic đánh giá vượt quá chi phí thực tế => Không admissible.
- Do không admissible nên chắc chắn Không consistent (vì consistent luôn kéo theo admissible khi $h(G) = 0$).
- Tích chọn:
  - Admissible: [x] Không
  - Consistent: [x] Không

d) Heuristic h_2 có admissible không? Có consistent không?

- So sánh $h_2(n)$ với $h^*(n)$:
  - $A: h_2 = 11 \leq 11$
  - $B: h_2 = 7 \leq 9$

- $C: h_2 = 6 \leq 7$
- $D: h_2 = 4 \leq 6$
- $E: h_2 = 5 \leq 5$
- $F: h_2 = 1 \leq 2$
- $G: h_2 = 0 \leq 0$
- Với mọi nút n, ta đều có $h_2(n) \leq h^*(n) \implies$ Admissible.

- Kiểm tra điều kiện consistent:$| h_2(u) - h_2(v) |$ <= c(u,v) trên từng cạnh:
  - Xét cạnh $A - B$ ($c = 3$):| 11 - 7 | = 4 > 3.
    - Đi từ $B \rightarrow A$: h_2 = 11 > c(B, A) + h_2(B) = 3 + 7 = 10 => Vi phạm tính consistent.
  - Xét cạnh $A - C$ ($c = 4$): | 11 - 6 | = 5 > 4.
    - Đi từ $C \rightarrow A$: h_2(A) = 11 > c(C, A) + h_2(C) = 4 + 6 = 10 => Vi phạm tính consistent.
- Tích chọn:
  - Admissible: [x] Có
  - Consistent: [x] Không

Bài 9. Trắc nghiệm và trả lời ngắn về tìm kiếm

a) Đánh dấu Đúng/Sai

1. Depth-First Tree Search trên một đồ thị hữu hạn luôn được đảm bảo là complete.

- Đáp án: [X] Sai
- *Giải thích:* Trong Tree Search không lưu tập trạng thái đã duyệt (explored set). Nếu đồ thị hữu hạn có chứa chu trình (cycle), DFS có thể đi vào một vòng lặp vô hạn và không bao giờ tìm thấy đích.

2. Breadth-First Tree Search trên một đồ thị hữu hạn luôn được đảm bảo là complete.

- Đáp án: [X] Đúng
- *Giải thích:* BFS tìm kiếm theo từng tầng độ sâu tăng dần (d = 0, 1, 2,...). Nếu tồn tại nghiệm ở độ sâu hữu hạn d, BFS chắc chắn sẽ tìm ra nghiệm đó.

3. Iterative Deepening Tree Search trên một đồ thị hữu hạn luôn được đảm bảo là complete.

- Đáp án: [X] Đúng

- *Giải thích:* IDS kết hợp ưu điểm của BFS và DFS bằng cách tăng dần giới hạn độ sâu cắt tỉa, do đó nếu có nghiệm ở độ sâu hữu hạn, IDS luôn luôn tìm ra nghiệm.

4. Với mọi đồ thị không có chu trình, Graph Search luôn có frontier lớn hơn Tree Search.

- Đáp án: [X] Sai
- *Giải thích:* Graph Search loại bỏ các trạng thái trùng lặp thông qua tập explored, do đó kích thước frontier của Graph Search luôn nhỏ hơn hoặc bằng (không bao giờ lớn hơn) so với Tree Search.

5. Iterative Deepening Graph Search có độ phức tạp thời gian của BFS và độ phức tạp bộ nhớ của DFS.

- Đáp án: [X] Sai
- *Giải thích:* Khẳng định trên chỉ đúng với Iterative Deepening Tree Search (bộ nhớ $O(bd)$ của DFS và thời gian $O(b^d)$ của BFS). Nếu áp dụng Graph Search cho Iterative Deepening, ta bắt buộc phải lưu tập explored của toàn bộ các nút đã mở rộng, khi đó độ phức tạp bộ nhớ sẽ tương đương BFS ($O(b^d)$) chứ không còn là bộ nhớ nhỏ tuyến tính của DFS.

6. Nếu $h_1(s)$ và $h_2(s)$ đều là heuristic consistent thì $\min(h_1(s), h_2(s))$ cũng nhất thiết consistent.

- Đáp án: [X] Sai
- *Giải thích:* Phép lấy giá trị lớn nhất $\max(h_1, h_2)$ luôn đảm bảo tính consistent, nhưng phép lấy giá trị nhỏ nhất $\min(h_1, h_2)$ không bảo toàn tính chất consistent (nó có thể làm tăng độ chênh lệch giữa hai nút kề nhau vượt quá chi phí cạnh).

b) Điền các giá trị heuristic sao cho admissible nhưng không consistent

1. Phân tích đồ thị để tính chi phí thực tế $h^*(s)$:

- G là đích => h*(G) = 0.
- $A \to G$: cạnh có trọng số 3 => h^*(A) = 3.
- $C \to G$: cạnh có trọng số 2 => h*(C) = 2.
- $S \to G$:
    - Qua nhánh A: $S \to A \to G = 1 + 3 = 4$

- Qua nhánh C: $S \to C \to G = 2 + 2 = 4$
  => h*(S) = 4.

2. Điều kiện cần thỏa mãn:

- Admissible: $h(s) \leq h^*(s)$ với mọi s.
  - Cần: $h(S) \leq 4, h(A) \leq 3, h(C) \leq 2, h(G) = 0$.
- Không consistent: Tồn tại ít nhất một cạnh (u, v) có trọng số $c(u, v)$ sao cho $h(u) > c(u, v) + h(v)$.

3. Đề xuất bộ giá trị:

- Đặt $h(S) = 4$ (thỏa mãn $4 \leq h^*(S) = 4$).
- Đặt $h(A) = 1$ (thỏa mãn $1 \leq h^*(A) = 3$).
- Đặt $h(C) = 2$ (thỏa mãn $2 \leq h^*(C) = 2$).
- Đặt $h(G) = 0$.

Kiểm tra vi phạm Consistent: Xét cạnh $S \to A$ có chi phí $c(S, A) = 1$:

$$h(S) = 4$$

$$c(S, A) + h(A) = 1 + 1 = 2$$

Vì $4 > 2$ ($h(S) > c(S, A) + h(A)$) nên vi phạm tính consistent.

Bảng giá trị

Trạng thái (State)	h(s)
S	4
A	1
C	2
G	0